

Analyse de la composante principale et reconnaissance faciale

Qu'est ce que c'est?

- Moyen de reconnaissance d'un visage sur une image et pouvoir le comparer à une base de données.
- Domaines d'utilisation: Police, douane, services de sécurité...

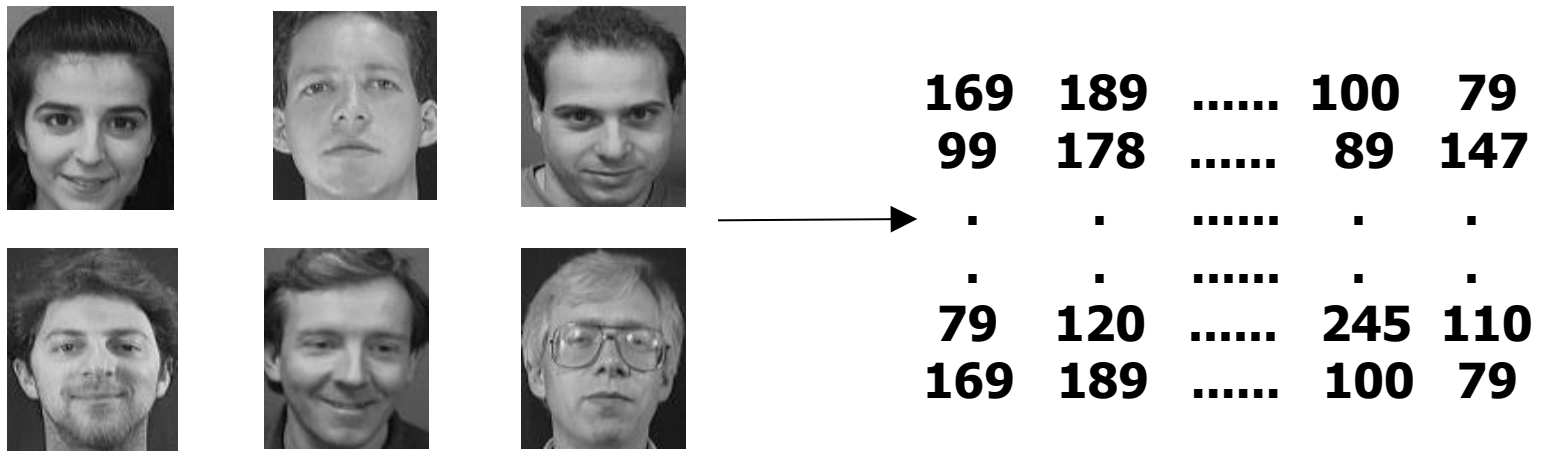
Comment ça marche?

Différentes photos de la base de données



Calcul des images

Transformation des photos en une matrice des images. Chaque colonne sera équivalente à une photo de la base de données.



Transformation Image en vecteur

$$I_i = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1N} \\ a_{21} & a_{22} & \dots & a_{2N} \\ \vdots & \vdots & \ddots & \vdots \\ a_{N1} & a_{N2} & \dots & a_{NN} \end{bmatrix}_{N \times N} \xrightarrow{\text{concatenation}} \begin{bmatrix} a_{11} \\ \vdots \\ a_{1N} \\ \vdots \\ a_{2N} \\ \vdots \\ a_{NN} \end{bmatrix}_{N^2 \times 1} = \Gamma_i$$

Le visage moyen est déduit des M visages d'apprentissage. Il traduit les caractéristiques communes à tous ces visages.

$$\Psi = \frac{1}{M} \sum_{i=1}^M \Gamma_i$$



AUSTRIA



AFGHANISTAN



ARGENTINA



BURMA (MYANMAR)



GERMANY



GREECE



CAMBODIA



ENGLAND



ETHIOPIA



FRANCE



IRAQ



IRELAND



MONGOLIA



PERU



POLAND



PUERTO RICO



UZBEKISTAN



AFRICAN AMERICAN

Calcul des poids associés à chaque visages propres

Le visage moyen est soustrait des visages d'apprentissage, ce qui ne laisse alors que les informations propres à chaque visages de référence.

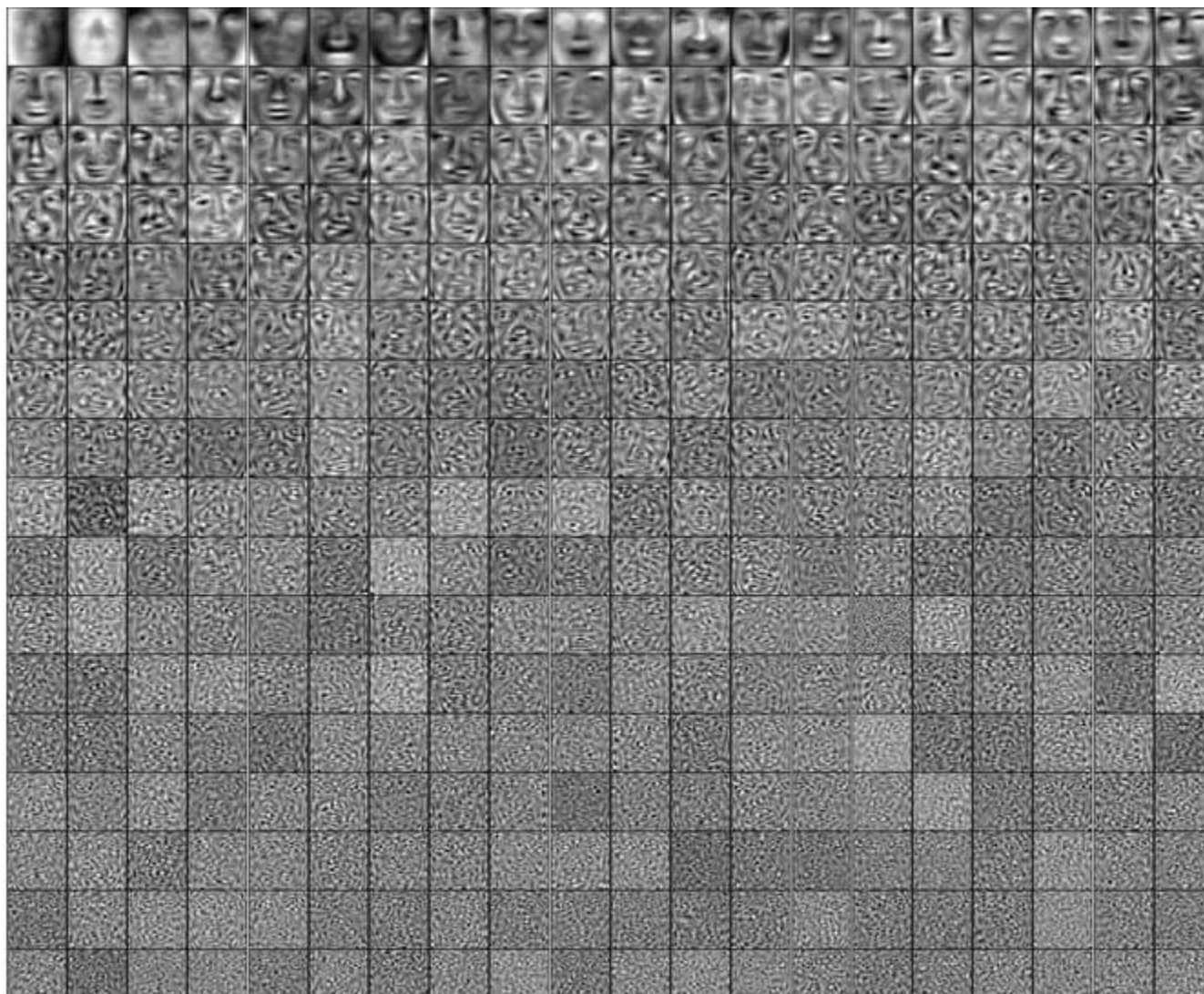
$$\Phi_i = \Gamma_i - \Psi$$

Poids associés aux visages propres à partir des caractéristiques propres d'une image de référence (ou d'apprentissage)

$$\Phi_i = \sum_{j=1}^K w_j u_j$$

$$w_j = u_j^T \Phi_i$$

EigenFace



Les étapes en bref

Computation of the eigenfaces

Step 1: obtain face images $I_1, I_2, \dots, I_M$ (training faces)

(**very important:** the face images must be *centered* and of the same *size*)

Step 2: represent every image I_i as a vector Γ_i

Step 3: compute the average face vector Ψ : $\Psi = \frac{1}{M} \sum_{i=1}^M \Gamma_i$

Step 4: subtract the mean face: $\Phi_i = \Gamma_i - \Psi$

Step 5: compute the covariance matrix C :

$$C = \frac{1}{M} \sum_{n=1}^M \Phi_n \Phi_n^T = AA^T \quad (N^2 \times N^2 \text{ matrix})$$

$$\text{where } A = [\Phi_1 \ \Phi_2 \ \dots \ \Phi_M] \quad (N^2 \times M \text{ matrix})$$

Step 6: compute the eigenvectors u_i of AA^T

Calcul des composantes principales

```
from sklearn.decomposition import PCA
```

```
# Compute a PCA (eigenfaces) on the olivetti  
dataset
```

```
# dimensionality reduction
```

```
n_components = 150
```

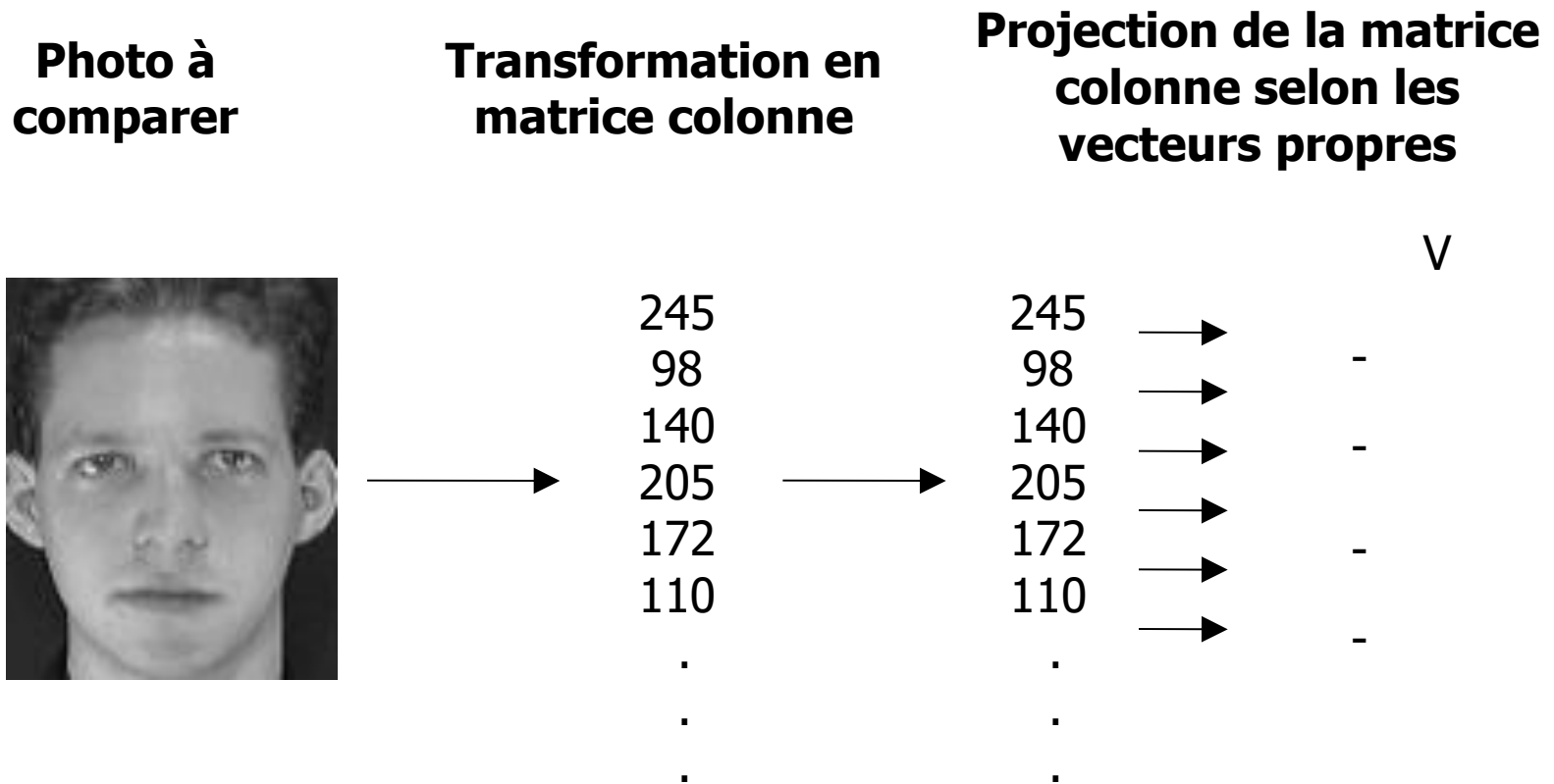
```
pca = PCA(n_components=n_components,  
          svd_solver='randomized',  
          whiten=True).fit(Xtrain)
```

3- Photo à comparer: prenons par exemple la photo suivante d'une personne de la base de donnée mais sous un autre aspect.

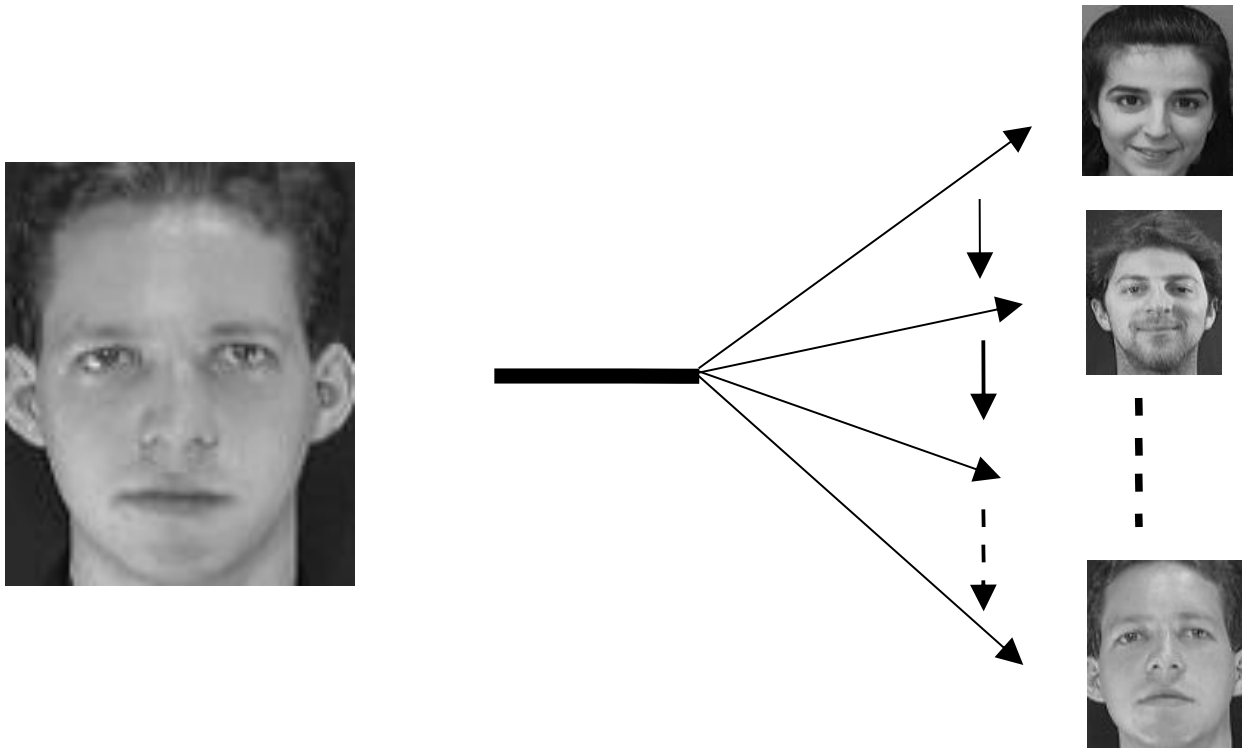


Le but consistera à ressortir de la base de données la personne ressemblant le plus à notre image cible.

Projection de la photo selon les mêmes caractéristiques tirées précédemment.



4- Comparaison: nous calculons la différence entre la projection de la photo cible et les projections de la base de données.



Résultat:

Une fois la comparaison terminée nous ressortons la photos avec laquelle nous obtenons la plus petite différence.

Image de départ



Même
personne

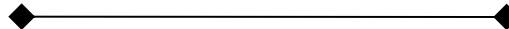


Image retrouvée



```
from sklearn.datasets import fetch_olivetti_faces

# Charger la base de données Olivetti
olivetti = fetch_olivetti_faces()

# Afficher les caractéristiques de la base de données
print("Images : ", olivetti.images.shape)
print("Labels : ", olivetti.target.shape)

import numpy as np
import matplotlib.pyplot as plt

# Calculer le visage moyen
mean_face = np.mean(olivetti.images, axis=0)

# Afficher le visage moyen
plt.imshow(mean_face, cmap='gray')
plt.title('Visage moyen')
plt.show()
```



```
# Retrancher Le visage moyen de chaque image  
X = olivetti.images - mean_face  
  
# Afficher Les dimensions de la matrice des faces après la normalisation  
print("Dimensions de la matrice des faces normalisées : ", X.shape)
```

```
from sklearn.decomposition import PCA  
# Appliquer la méthode PCA  
X = olivetti.images.reshape((len(olivetti.images), -1))  
  
n_components = 10  
pca = PCA(n_components=n_components)  
pca.fit(X)
```

```
# Afficher les 10 premières EigenFaces
for i in range(10):
    plt.imshow(pca.components_[i].reshape((64, 64)), cmap='gray')
    plt.title('EigenFace {}'.format(i+1))
    plt.show()
```

```
# Projeter chaque image sur les composantes principales
X_proj = pca.transform(X)

# Afficher la taille de la matrice des projections
print("Taille de la matrice des projections :", X_proj.shape)
```

```
# Choix d'une image requête
img_req = X[0]
img_2d = img_req.reshape((64, 64))
plt.imshow(img_2d, cmap='gray')
plt.axis('off')
plt.title("Image requête")
plt.show()
```

```
# Calcul des projections de l'image requête sur les composantes principales retenues
pca = PCA(n_components=10)
pca.fit(X)
img_req_proj = pca.transform(img_req.reshape(1, -1))
```

```
# Calcul de la distance entre le vecteur projection de l'image requête
distances = np.linalg.norm(X_proj - img_req_proj, axis=1)
```

```
# Afficher l'image la plus proche  
img_proche = X[indice_min,:].reshape(64,64)  
plt.imshow(img_proche, cmap='gray')  
plt.title('Image la plus proche')  
plt.show()
```

